

Rail Corrugation — Approach Write-Up

Subsystem: PS3, Rail Corrugation (Train Condition Monitoring)

Task: 3-class classification of 1-second axle-box vibration/shock recordings into Normal / Side I / Side II.

1. Data & Train/Validation Split

Data provided: 272 labelled training files (Train1.csv–Train272.csv), each 10,000 rows $\times$ 129 columns (1 rotating-speed column + 64 axle-box positions $\times$ {vibration, shock} = 128 signal channels), sampled at 10,000 Hz for 1 second. Per the Rail Corrugation Info Kit, positions 1/3/5/7 correspond to Side I and positions 2/4/6/8 to Side II. Labels are severely imbalanced: 234 Normal / 14 Side I / 24 Side II.

Assumption (design decision left open by the spec): the top-level PS3 spec explicitly leaves the train/validation split strategy to each team, provided it's justified (Section 2.3/3.2). We chose **5-fold Stratified K-Fold** over a single train/test holdout for one reason: with only 14 Side I examples total, a single holdout split risks an unrepresentative validation set (e.g. 0–2 Side I examples), making any single score unreliable. Stratified K-Fold forces every fold to preserve the same ~86%/5%/9% class ratio as the full dataset, and every file is used as validation exactly once across the 5 folds, so the reported score is an average over the entire dataset rather than one arbitrary split. This choice directly addresses the "valid train/validation splits, no data leakage" criterion the spec calls out as being judged alongside the headline metric (Section 3.2).

We additionally ran a **nested cross-validation** (5-fold outer / 3-fold inner) for hyperparameter tuning, so that any hyperparameter search never sees the fold it is later scored on — avoiding the optimistic bias that would result from tuning and evaluating on the same folds.

2. Feature Engineering

Each file's 128 signal channels are processed independently — car and position are parsed directly out of each column's own header text via regex (e.g. "Shock of bearing in position 3 of car 5"), rather than assumed from a fixed column order, so a reordered or partial column set doesn't silently misassign a channel to the wrong car/side.

For each channel (plus the rotating-speed column), we extract 9 time- and frequency-domain statistics: mean, standard deviation, RMS, peak-to-peak, crest factor, kurtosis, dominant FFT frequency, spectral energy, and spectral centroid. This gives $128 \times 9 + 9 = 1,161$ raw features per file.

Design decision: we deliberately kept full per-car/per-position detail rather than aggregating statistics across the 8 cars (e.g. taking a mean/max over cars). The reasoning: a corrugation fault is localized to a specific car and side, and aggregating across cars risks averaging away the one car that actually carries the fault signature. Dimensionality is instead handled downstream via SelectKBest (ANOVA F-test) inside the modelling pipeline, refit independently on each training fold — so the 1,161 features are narrowed to the ~50 most class-discriminative ones without ever seeing the validation fold.

Known limitation: 38 of 272 training files have a near-constant rotating-speed signal, which makes kurtosis mathematically undefined (division by a near-zero fourth central moment) and produces NaN. These are filled with 0 before modelling. We did not investigate whether 0 is the ideal fill value for this specific case, only that it

prevents a fold-fitting crash while affecting only one of 1,161 features (`speed_kurtosis`).

3. Class Imbalance Handling

Given the 234/14/24 split, five separate mechanisms were used, all weighting-based rather than resampling-based:

- 1 `class_weight='balanced'` (or `'balanced_subsample'` for Random Forest) on 5 of 6 candidate models, so a mistake on a rare Side I example is penalised more than a mistake on a common Normal example.
- 2 For XGBoost, which doesn't support `class_weight` and whose usual imbalance mechanism (`scale_pos_weight`) is binary-only, we instead compute `sample_weight` via `compute_sample_weight('balanced', ...)` fresh on each fold's training portion only — functionally equivalent, computed manually because the built-in option doesn't support 3 classes.
- 3 `StratifiedKFold` (not plain `KFold`) in every cross-validation loop in the project, for the reason given in Section 1.
- 4 Random Forest's and Decision Tree's `min_samples_leaf/min_samples_split` were loosened relative to an earlier, shared baseline configuration (`min_samples_leaf` 10→3 and 15→5 respectively), which had been sized for a ~700-row dataset and made it near-impossible for either tree to isolate a leaf built from Side I's 14 examples.
- 5 Macro F1 (Section 4) as the model-selection metric itself, which is the umbrella reason the above mechanisms matter — accuracy would let a model that always predicts Normal score ~86% while never catching a single fault.

Not yet attempted: oversampling (e.g. SMOTE) to synthesize additional Side I training examples, and an ensemble-based resampling method (`EasyEnsembleClassifier`, present in the model registry but currently commented out / unused). These remain a natural next step, since Side I's F1 is bottlenecked partly by raw scarcity (Section 5) that weighting alone cannot fix — weighting changes how much an existing example counts, not how much information the model actually gets to learn from.

4. Model Selection & Metric

Metric: macro F1 — the unweighted average of each class's own F1 score. This is the metric fixed by the hackathon's own scoring rubric (Section 5.2 of the top-level PS3 spec) and is the correct choice independent of that requirement: with 86% of files labelled Normal, plain accuracy would reward a model that ignores both fault classes entirely. Macro F1 forces the rare classes to matter equally to the common one.

Models compared: LASSO and Ridge (L1/L2-regularised logistic regression), Random Forest, Decision Tree, XGBoost, and SVM (RBF kernel) — six candidates chosen to span linear, tree-ensemble, and margin-based approaches, since with 1,161 correlated features over only 272 rows it wasn't obvious in advance which inductive bias would fit best.

We evaluated in three stages:

Stage	Method	Purpose
1	Flat 5-fold CV, default hyperparameters	Fast first-pass comparison
2	Nested CV (5-fold outer / 3-fold inner, GridSearchCV)	Unbiased score under tuned hyperparameters
3	Nested CV repeated at 3 random seeds (42, 7, 123)	Stability check — does the "best model" hold up under a different fold split?

Stage 1 selected **SVM** (macro F1 = 0.655). Stage 2, at a single seed, selected **LASSO** (0.7094) instead. Stage 3 is why we didn't stop at Stage 2: repeating the exact same nested-CV procedure at two more seeds showed LASSO's score swinging from 0.573 to 0.709 — a range of 0.136 — while SVM's swung only from 0.607 to 0.675 (range 0.068), roughly half the spread, with a higher mean across the three seeds (0.6415 vs. 0.6337). Every model except Ridge, Decision Tree, and XGBoost "won" at least one of the three seeds, which confirms the ranking genuinely depends on which files land in which fold when the minority class is this small — a single-seed result is not sufficient evidence to select a final model here.

Final model: SVM (RBF kernel, `class_weight='balanced'`), selected for having both the best mean macro F1 and the lowest run-to-run variance across seeds — prioritising a score we can trust to hold on the real held-out test set over a higher but less reproducible one.

5. Diagnostic Findings (Per-Class Breakdown)

Out-of-fold per-class results for the selected SVM model (leak-free, same 5-fold split as Stage 1):

Class	Precision	Recall	F1	Support
Normal	0.962	0.876	0.917	234
Side I	0.200	0.429	0.273	14
Side II	0.690	0.833	0.755	24

Side I is the clear weak point. The confusion matrix shows this is driven by confusion with **Normal**, not with Side II: of 14 true Side I files, 7 are missed and predicted Normal (only 1 is confused with Side II); conversely, of 30 files predicted Side I, 21 are actually Normal. This points to two compounding causes: (a) with only 14 real examples, the model has not seen enough variety of what a mild Side I fault looks like to draw a stable boundary against Normal, and (b) `class_weight='balanced'` deliberately trades precision for recall on the rare class, which is why recall (0.429) is higher than precision (0.200) rather than both being uniformly low.

6. Other Assumptions Made

- **Environment/reproducibility:** every model and CV split uses `random_state=42` (or the seed under test in the stability sweep), so all reported numbers are exactly reproducible from the raw training files.
- **No test-set contamination:** the 68-file held-out test set (Test1.csv–Test68.csv) was downloaded but not used in any of the analysis above — all comparisons above are within the 272 labelled training files only, via cross-validation.

- **Spec interpretation:** an earlier version of the Info Kit referenced a required `predict.py` CLI script, which appears to have been superseded by the top-level spec's app-only submission requirement (Section 4.1) — we are proceeding on the assumption that only the app is required, not a CLI, pending confirmation against the Info Kit's current text.

7. Status / Next Steps

Modelling and evaluation are complete for this subsystem; the app (compulsory deliverable, Section 4.1 item 3) is the remaining item before `predictions.zip` can be generated by running the real test files through it. Two modelling improvements identified but not yet implemented: (1) SMOTE oversampling for Side I specifically, and (2) a per-car "side asymmetry" feature (Side I stat – Side II stat, per car) to give the model a more direct signal for a lopsided/faulty car, rather than requiring it to infer that from 1,161 largely independent absolute-value features.